Cognitive Robotics report

Awab Akram

March 27, 2025

1 Literature Review

Navigation is a fundamental capability in robotics, enabling agents to move purposefully through environments to reach specific goals. Traditional navigation pipelines often rely on SLAM, path planning, and control modules. However, recent advances in deep learning have introduced end-to-end or hybrid approaches that improve robustness and adaptability in complex environments.

1.1 End-to-End Learning

End-to-end learning has emerged as a powerful paradigm in robotic navigation, enabling robots to learn complex behaviours directly from raw sensory inputs. This approach bypasses the need for hand-engineered features or intermediate representations, allowing for more robust and adaptable navigation systems.

One of the key advantages of end-to-end learning is its ability to learn directly from experience. As Ma et al. (2025) note, this allows robots to reason about traversability beyond simple geometric considerations, accounting for the physical outcomes of their actions and exploiting patterns in real-world environments. This experiential learning can lead to more nuanced and effective navigation strategies, particularly in complex and dynamic environments.

Deep reinforcement learning (DRL) has been a driving force behind many end-to-end navigation systems. Levine et al. (2016) demonstrated the potential of end-to-end DRL for robotic control, showing that policies could be learned to map raw visual input directly to motor torques. This approach has since been applied to various navigation tasks, with researchers exploring different network architectures and training methodologies to improve performance and generalization.

Graph Neural Networks (GNNs) have also shown promise in end-to-end navigation learning, particularly for multi-robot systems. Jiang et al. (2023) used a GNN combined with a convolutional neural network to learn decentralized control policies for multi-robot formations, operating directly on robot sensor observations. This approach demonstrates the potential for scalable, end-to-end learning of complex multi-robot behaviours.

However, end-to-end learning is not without its challenges. Training such systems often requires large amounts of data and computational resources. Additionally, the "black box" nature of deep neural networks can make it difficult to interpret or debug learned behaviours. To address these issues, some researchers have explored hybrid approaches that combine end-to-end learning with more traditional navigation methods.

Despite these challenges, the potential benefits of end-to-end learning for robotic navigation are significant. As Singh et al. (2019) argue, by enabling robots to improve their skills directly in real-world environments without manual reward design, end-to-end learning represents a step towards enabling lifelong learning for robotic systems "in the wild". This capability could lead to more versatile and adaptable robots capable of operating in a wide range of environments and scenarios.

1.2 Hybrid methods

Hybrid approaches in robotic navigation combine classical methods with modern machine learning techniques to leverage the strengths of both paradigms. These methods aim to enhance navigation performance while mitigating the limitations of purely classical or purely learning-based approaches.

One notable hybrid strategy involves using deep neural networks to augment classical navigation pipelines. For instance, researchers have proposed combining classical simultaneous localization and mapping (SLAM) techniques with deep learning components (Wani et al., 2020). This approach allows for more accurate mapping and localization while leveraging the adaptability of learned models for dynamic environments.

In addition, reinforcement learning has been used in hybrid methods to improve local path planning in dynamic scenes. By combining reinforcement learning with traditional algorithms like A* and Adaptive Monte Carlo Localization (AMCL), researchers have developed navigation systems that can adapt to changing environments and avoid obstacles effectively (Zhao and Ning, 2022).

2 Experiment and Methodology

To investigate the effect of deep learning architecture and hyperparameter tuning on visual recognition tasks, a series of experiments were conducted using the CIFAR-100 dataset. This dataset was selected due to its greater complexity compared to alternatives like MNIST, as it contains 100 fine-grained object categories with 600 images each (500 training, 100 testing). All input images were resized to 32×32 RGB and normalized to the [0, 1] range. Labels were one-hot encoded for classification using the softmax activation function.

2.1 Model Architecture

The base model architecture consisted of up to three convolutional blocks, each comprising two convolutional layers (with ReLU activation), batch normalization, max pooling, and dropout. The number of convolutional blocks was determined dynamically based on the filter configuration suggested during hyperparameter tuning. The convolutional stack was followed by a flattening layer, a fully connected dense layer, dropout, and a final softmax classification layer outputting 100 class probabilities.

This flexible design allowed for architectural variation in both depth and width, enabling exploration of lightweight to moderately deep CNNs. Dropout was applied after each pooling layer and dense layer to mitigate overfitting.

2.2 Hyperparameter Optimisation

Optuna, a hyperparameter optimization framework, was used to explore a predefined search space across 23 trials. Each trial involved training a distinct model configuration using a combination of:

- Learning rate: 1e-5, 1e-4, 1e-3
- Optimizer: Adam, RMSprop
- Dropout rate: 0.2, 0.3, 0.4
- Convolutional filters per block:
  - Block 1: 64, 128, 256 (always included)
  - Block 2: 0, 64, 128, 256
  - Block 3: 0, 64, 128, 256
- Dense units: 128, 256, 512
- Batch size: 64, 128

Training was performed for up to 50 epochs using the categorical cross-entropy loss function and early stopping based on validation loss.

2.3 Training and Evaluation

Models were trained on the CIFAR-100 dataset using GPU acceleration where available. Each trial recorded validation accuracy, test accuracy, and model characteristics. To avoid overfitting and ensure efficient training, early stopping was implemented to monitor the validation loss with a patience of 10 epochs. A shorter patience of 5 was initially tested but resulted in training terminating too early, often before convergence. The restore_best_weights setting was used to ensure that the final model reflected the best-performing epoch on the validation set.

Additionally, to address challenges with model convergence, ReduceLROnPlateau was applied to reduce the learning rate dynamically when validation loss plateaued. This was particularly important because larger learning rates (e.g. 1e-2) were found to degrade performance severely, often resulting in validation accuracies below 10%. Rather than defaulting to an extremely small initial learning rate like 1e-6, this adaptive strategy allowed training to begin more aggressively while still enabling refinement over time.

Some models reached the 50-epoch cap without triggering early stopping, suggesting that additional training time could have improved performance further. However, the limit was maintained to ensure time-efficient experimentation.

3 Results and Analysis

The hyperparameter tuning process provided insights into how different architectural and training choices influenced model performance on the CIFAR-100 dataset.

A validation set comprising 20% of the training data was used to monitor the model's ability to generalise to unseen examples during training. Validation accuracy served as a key indicator for hyperparameter tuning, enabling detection of overfitting and early performance saturation.

Across 23 trials, validation and test accuracy varied widely, ranging from as low as 12.9% to as high as 58.75%. This variation highlighted the sensitivity of CNNs to both structural and training parameters.

3.1 Effect of Hyperparameters

3.1.1 Dropout rate

Dropout rate played a key role in controlling overfitting. Models with a lower dropout rate of 0.2 generally outperformed those with higher rates, suggesting that excessive regularization hindered the network's capacity to learn complex features in the high-dimensional CIFAR-100 dataset. The top-performing model used a dropout rate of 0.2.

3.1.2 Convolutional layers

Convolutional depth and filter size were strongly correlated with accuracy. Deeper models with multiple convolutional blocks and high filter counts (e.g. 256 in each block) consistently outperformed shallower counterparts. Trials that excluded later convolutional layers (e.g. filters_2 or filters_3 set to 0) typically saw a drop in performance, indicating the importance of deeper hierarchical feature extraction for fine-grained object classification.

3.1.3 Dense layer size

Dense units also contributed to performance, with 512 units offering the best generalisation. Lower values like 128 or 256 resulted in reduced accuracy in most cases, particularly when paired with deeper CNN stacks.

3.1.4 Learning rate

Learning rate had a pronounced effect on stability and convergence. The most successful models used a learning rate of 0.0001, which provided a stable training curve and allowed for fine-tuned adjustments. In theory, lower learning rates should result in better convergence; however, due to the 50-epoch cap, these models may not have had adequate time to train fully. Higher learning rates (e.g. 0.001) occasionally achieved strong validation accuracy but were less consistent on the test set, suggesting overfitting or unstable convergence.

3.1.5 Optimiser

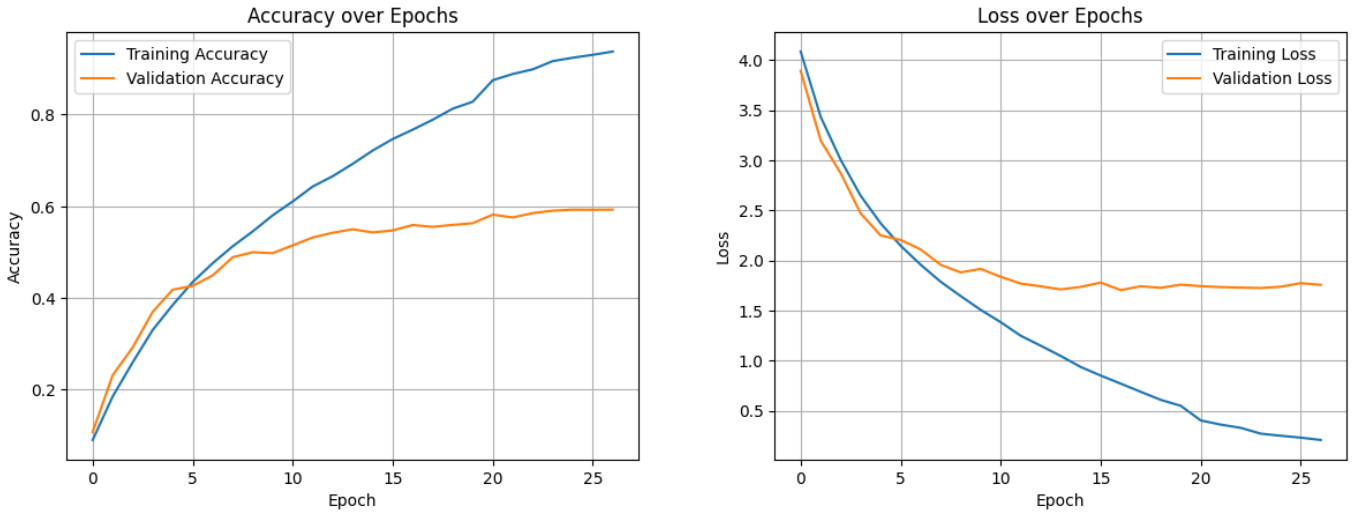
Optimiser choice revealed RMSprop to be more effective than Adam in this context. The best-performing trials all used RMSprop, likely due to its more adaptive learning rate handling in CNNs for image data.

3.2 Best Model Evaluation

The highest-performing model achieved a validation accuracy of 59.71% and a test accuracy of 58.75% after 30 epochs. This configuration used three convolutional blocks with 256 filters each, a dropout rate of 0.2, a dense layer of 512 units, and RMSprop with a learning rate of 0.0001.

Several other models reached the 50-epoch cap without triggering early stopping, indicating they may have improved further with longer training — though this was limited for time efficiency.

The model was retrained using the best hyperparameters and evaluated on the held-out test set. The final test accuracy of 58.79% reflects strong generalisation, closely matching validation performance. Figures 1 and 2 show the training and validation accuracy and loss curves across epochs. These indicate steady learning with no major overfitting, further supporting the model's robustness.



(a) Accuracy over epochs

(b) Loss over epochs

Figure 1: Training and validation accuracy and loss for the best model

4 Conclusion

This project explored the application of convolutional neural networks (CNNs) to the CIFAR-100 dataset, with a focus on how hyperparameter choices affect classification performance. Through systematic tuning using Optuna, the study revealed that lower dropout rates, deeper architectures with high filter counts, and careful selection of learning rate and optimizer all contributed significantly to improved accuracy. RMSprop with a learning rate of 0.0001 consistently outperformed other configurations, and models with three convolutional blocks of 256 filters achieved the best results.

While the models achieved strong performance relative to the dataset's difficulty, there is still room for improvement. Future work could explore transfer learning with pretrained architectures such as ResNet or EfficientNet, and implement data augmentation strategies to increase robustness. Additionally, integrating techniques such as Grad-CAM or feature map visualisation could offer interpretability insights, bridging the gap between performance and transparency in robotic vision systems.

Ultimately, this investigation demonstrates how even relatively lightweight deep learning models, when carefully tuned, can achieve competitive results on challenging visual recognition tasks, laying the groundwork for more capable perception systems in robotic applications.

References

Chao Jiang, Xinchi Huang, and Yi Guo. End-to-end decentralized formation control using a graph neural network-based learning method. *Frontiers in Robotics and AI*, 10, 2023. ISSN 2296-9144. doi: 10.3389/frobt.2023.1285412. URL `https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2023.1285412`.

Sergey Levine, Chelsea Finn, Trevor Darrell, and Pieter Abbeel. End-to-end training of deep visuomotor policies. 2016. URL `https://arxiv.org/abs/1504.00702`.

Song Ma, Jingqing Ruan, Yali Du, Richard Bucknall, and Yuanchang Liu. An end-to-end deep reinforcement learning based modular task allocation framework for autonomous mobile systems. *IEEE Transactions on Automation Science and Engineering*, 22:1519–1533, 2025. doi: 10.1109/TASE.2024.3367237.

Avi Singh, Larry Yang, Kristian Hartikainen, Chelsea Finn, and Sergey Levine. End-to-end robotic reinforcement learning without reward engineering. 2019. URL `https://arxiv.org/abs/1904.07854`.

Saim Wani, Shivansh Patel, Unnat Jain, Angel X. Chang, and Manolis Savva. Multion: Benchmarking semantic map memory using multi-object navigation. 2020. URL `https://arxiv.org/abs/2012.03912`.

Kaidong Zhao and Li Ning. Hybrid navigation method for multiple robots facing dynamic obstacles. *Tsinghua Science and Technology*, 27(6):894–901, 2022. doi: 10.26599/TST.2021.9010073.

Appendix A Main Training Code

```
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import cifar100
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout,
    BatchNormalization
from tensorflow.keras.optimizers import Adam, RMSprop
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
import tensorflow as tf


print("Num GPUs Available:", len(tf.config.list_physical_devices('GPU')))

BATCH_SIZE = 64
EPOCHS = 50
LEARNING_RATE = 0.0001
VALIDATION_SPLIT = 0.2
NUM_CLASSES = 100
INPUT_SHAPE = (32, 32, 3)
(x_train, y_train), (x_test, y_test) = cifar100.load_data(label_mode='fine')

# Normalize pixel values
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

y_train_cat = to_categorical(y_train, NUM_CLASSES)
y_test_cat = to_categorical(y_test, NUM_CLASSES)

print(f"Training samples: {x_train.shape[0]}, Test samples: {x_test.shape[0]}")
model = Sequential()

model.add(Conv2D(256, (3, 3), padding="same", activation='relu', input_shape=INPUT_SHAPE))
model.add(BatchNormalization())
model.add(Conv2D(256, (3, 3), padding="same", activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.2))

model.add(Conv2D(256, (3, 3), padding="same", activation='relu'))
model.add(BatchNormalization())
model.add(Conv2D(256, (3, 3), padding="same", activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.2))

model.add(Conv2D(256, (3, 3), padding="same", activation='relu'))
model.add(BatchNormalization())
model.add(Conv2D(256, (3, 3), padding="same", activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.2))

# Dense Block
model.add(Flatten())
model.add(Dense(512, activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(NUM_CLASSES, activation='softmax'))

model.summary()
model.compile(optimizer=RMSprop(learning_rate=LEARNING_RATE),
              loss='categorical_crossentropy',
              metrics=['accuracy'])
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True  # Restore weights from the best epoch
)

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,              # Reduce by half
    patience=3,              # Wait 3 epochs with no improvement
    min_lr=1e-6,             # Don't go below this
    verbose=1
)
history = model.fit(x_train, y_train_cat,
                    validation_split=VALIDATION_SPLIT,
                    epochs=EPOCHS,
                    batch_size=BATCH_SIZE,
                    verbose=1,
```

```
                    callbacks=[early_stopping, reduce_lr])

# Evaluate on test set
test_loss, test_acc = model.evaluate(x_test, y_test_cat, verbose=0)
print(f"Test Accuracy: {test_acc:.4f}")
# Plot training history
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.show()
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()
```

Listing 1: Main training loop and model definition

Appendix B Hyperparameter Tuning with Optuna

```
import optuna
import tensorflow as tf
from tensorflow.keras import backend as K
from tensorflow.keras.datasets import cifar100
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout,
    BatchNormalization
from tensorflow.keras.optimizers import Adam, RMSprop
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
import pandas as pd
import datetime
import io
import contextlib
import json


print("Num GPUs Available:", len(tf.config.list_physical_devices('GPU')))

NUM_CLASSES = 100
INPUT_SHAPE = (32, 32, 3)
MAX_EPOCHS = 50
VALIDATION_SPLIT = 0.2
(x_train, y_train), (x_test, y_test) = cifar100.load_data(label_mode='fine')

# Normalize pixel values
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

y_train_cat = to_categorical(y_train, NUM_CLASSES)
y_test_cat = to_categorical(y_test, NUM_CLASSES)
# Global list to store results
trial_log = []


def objective(trial):
    K.clear_session()

    lr = trial.suggest_categorical("lr", [1e-5, 1e-4, 1e-3])
    optimizer_name = trial.suggest_categorical("optimizer", ["Adam", "RMSprop"])
    dropout_rate = trial.suggest_categorical("dropout", [0.2, 0.3, 0.4])

    # Filters
    filters_1 = trial.suggest_categorical("filters_1", [64, 128, 256])  # always included
    filters_2 = trial.suggest_categorical("filters_2", [0, 64, 128, 256])
    filters_3 = trial.suggest_categorical("filters_3", [0, 64, 128, 256])

    dense_units = trial.suggest_categorical("dense_units", [128, 256, 512])
    batch_size = trial.suggest_categorical("batch_size", [64, 128])
```

```
    optimizer = Adam(learning_rate=lr) if optimizer_name == "Adam" else RMSprop(learning_rate=lr)

    print(f"\nTrial {trial.number} Hyperparameters:")
    print(f"  Learning rate: {lr}")
    print(f"  Optimizer:     {optimizer_name}")
    print(f"  Dropout:       {dropout_rate}")
    print(f"  Filters:       [{filters_1}, {filters_2}, {filters_3}]")
    print(f"  Dense units:   {dense_units}")
    print(f"  Batch size:    {batch_size}")


    model = Sequential()

    # Conv Block 1 (always included)
    model.add(Conv2D(filters_1, (3, 3), padding="same", activation='relu', input_shape=INPUT_SHAPE))
    model.add(BatchNormalization())
    model.add(Conv2D(filters_1, (3, 3), padding="same", activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(dropout_rate))

    # Conv Block 2 (optional)
    if filters_2 > 0:
        model.add(Conv2D(filters_2, (3, 3), padding="same", activation='relu'))
        model.add(BatchNormalization())
        model.add(Conv2D(filters_2, (3, 3), padding="same", activation='relu'))
        model.add(MaxPooling2D(pool_size=(2, 2)))
        model.add(Dropout(dropout_rate))

    # Conv Block 3 (optional)
    if filters_3 > 0:
        model.add(Conv2D(filters_3, (3, 3), padding="same", activation='relu'))
        model.add(BatchNormalization())
        model.add(Conv2D(filters_3, (3, 3), padding="same", activation='relu'))
        model.add(MaxPooling2D(pool_size=(2, 2)))
        model.add(Dropout(dropout_rate))

    # Dense block
    model.add(Flatten())
    model.add(Dense(dense_units, activation='relu'))
    model.add(Dropout(dropout_rate))
    model.add(Dense(NUM_CLASSES, activation='softmax'))

    model.compile(optimizer=optimizer,
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    # Callbacks
    early_stopping = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
    reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3, min_lr=1e-6, verbose
        =1)

    # Train the model
    history = model.fit(
        x_train, y_train_cat,
        validation_split=VALIDATION_SPLIT,
        epochs=MAX_EPOCHS,
        batch_size=batch_size,
        verbose=1,
        callbacks=[early_stopping, reduce_lr]
    )

    val_acc = history.history['val_accuracy'][-1]
    test_loss, test_acc = model.evaluate(x_test, y_test_cat, verbose=0)

    with io.StringIO() as buf, contextlib.redirect_stdout(buf):
        model.summary()
        model_summary_str = buf.getvalue()


    trial_log.append({
        'trial': trial.number,
        'lr': lr,
        'optimizer': optimizer_name,
        'dropout': dropout_rate,
        'filters_1': filters_1,
        'filters_2': filters_2,
        'filters_3': filters_3,
        'dense_units': dense_units,
        'batch_size': batch_size,
```

```
        'val_accuracy': val_acc,
        'test_accuracy': test_acc,
        'epochs_ran': len(history.history['accuracy']),
        'timestamp': datetime.datetime.now().isoformat(),
        'model_summary': model_summary_str
    })

    return val_acc

try:
    study = optuna.create_study(direction="maximize")
    study.optimize(objective, n_trials=25, show_progress_bar=True)

except KeyboardInterrupt:
    print("\nOptimization interrupted. Saving progress...")

finally:
    # Save to CSV and JSON no matter what
    pd.DataFrame(trial_log).to_csv("optuna_trials.csv", index=False)

    with open("optuna_trials_with_summaries.json", "w") as f:
        json.dump(trial_log, f, indent=2)

    print("Trials saved to 'optuna_trials.csv' and 'optuna_trials_with_summaries.json'")
```

Listing 2: Optuna tuning loop for CNN model